



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ
ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ
ΕΡΓΑΣΤΗΡΙΟ ΥΠΟΛΟΓΙΣΤΙΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

Εργαστήριο Λειτουργικών Συστημάτων
7ο Εξάμηνο, Ακαδημαϊκή περίοδος 2025-2026

Εργαστηριακή Άσκηση 2
Οδηγός Ασύρματου Δικτύου Αισθητήρων στο Λειτουργικό
Σύστημα Linux

oslab11

Αϊφαντή Ελισάβετ (03122821)

Βασίλειος Μιλτιάδης Πάντζος (03121281)

Περίληψη

Σκοπός της παρούσας εργαστηριακής άσκησης είναι η υλοποίηση κλήσεων συστημάτων για την ορθή λειτουργία ενός character driver που λαμβάνει δεδομένα από 16 διαφορετικούς αισθητήρες σχετικά με την θερμοκρασία, τη φωτεινότητα και το επίπεδο φόρτισής τους.

Κληθήκαμε να συμπληρώσουμε τις κλήσεις συστήματος `init`, `open`, `read`, `close` και `release`, καθώς και τις συναρτήσεις για `update` και `needs_refresh` των δομών `state` με σκοπό τη σωστή λήψη των δεδομένων για κάθε συσκευή και την αντίστοιχη μέτρησή της. Το πρόβλημα είναι ο ορθός χειρισμός των δομών δεδομένων του πυρήνα `file` και `inode`, η ορθή χρήση των δομών `sensor` και `state` ώστε να γίνει έγκυρη ανάγνωση των δεδομένων ενημερώνοντας τις σχετικές δομές και αποφεύγοντας το πρόβλημα των συνθηκών συναγωνισμού, όπως και ο ορθός χειρισμός της μνήμης για αποφυγή προβλημάτων ασφάλειας.

Εισαγωγή

Σε κάθε κλήση συστήματος αξιοποιούμε τα objects `inode` και `file`, τα οποία εμπεριέχουν pointers στις δομές που θέλουμε να χειριστούμε.

Το `inode` κάθε συσκευής δημιουργείται με τις εντολές `mknod` στο `mk-lunix-devs.sh`. Σε αυτό αποθηκεύονται metadata:

- `dev-t i_rdev`, το οποίο περιέχει το `major` και `minor` number της συσκευής
- `struct cdev *i_cdev`,

Η δομή `file` κάθε συσκευής δημιουργείται `implicitly` από τον `kernel` με την κλήση της `open`.

Σημειώνεται ότι χρησιμοποιούμε `per-device state` το `linux_sensor_struct` και ως `per-open state` το `linux_chrdev_state_struct`. Εδώ διακρίνουμε τη συσκευή από τον `reader`, καθώς ενώ μπορούμε να έχουμε πολλαπλές διεργασίες (`readers`) για μία συσκευή με δικό τους `buffer`, `buf_timestamp`, `buf_limit` και `σημαφόρο`, θα έχουμε μοναδική δομή `sensor` που τη διαμοιράζονται όλες οι διεργασίες. Κατέχει δικό της `spinlock` και `waitqueue` για να συγχρονίσει τους `readers`, και πίνακα `linux_msr_data_struct *msr_data[N_LUNIX_MSR]` για τις σελίδες στη μνήμη που αντιστοιχούν σε κάθε είδος μέτρησης.

Η ίδια η δομή `msr_data` θα περιέχει:

- Δικό της `timestamp`, το `last_update`, το οποίο συγκρίνεται με τα `buf_timestamp` της κάθε νέας μέτρησης των readers για να γίνει σωστά η ανανέωση των δεδομένων.
- Magic (sanity check value), που ορίζεται κατά την αρχικοποίηση του αισθητήρα ως `LUNIX_MSR_MAGIC`. Εάν γίνει έλεγχος και δεν έχει την τιμή αυτή, γνωρίζουμε ότι δεν έγινε αρχικοποίηση ή τουλάχιστον έγινε κάποιο memory corruption.
- Πίνακα `values[]`, ο οποίος περιέχει τις raw τιμές που λαμβάνει από το hardware. `values[0]` η πιο πρόσφατη ανάγνωση.

Syscall Init

Η `init` καλείται κατά την `insmod linux.ko`.

Με την `cdev_init` δημιουργούμε τη δομή `cdev` και τη συνδέουμε με τα file operations όπως έχουν οριστεί, για να αντιστοιχηθούν με τα system calls που μπορεί να κληθούν από τον χρήστη.

Λαμβάνουμε τα 48 device numbers με την κλήση της `int register_chrdev_region` (`dev_t first`, `unsigned int count`, `char *name`). Στο πρόγραμμα χρησιμοποιούμε τις παραμέτρους:

- `dev_t dev_no`, η οποία αποτελεί τον συνδυασμό `major` και `minor number`. Αρχικοποιείται με την `dev_no = MKDEV(LUNIX_CHRDEV_MAJOR, 0)`;
- `linux_sensor_cnt`, που στο header `linux.h` έχει οριστεί ως 16.
- “linux” ως όνομα συσκευής

Η παραπάνω συνάρτηση επιστρέφει 0 εάν είναι επιτυχής. Στην περίπτωση που αποτύχει, πρέπει να αναιρεθεί η ανάθεση αυτή με την `goto out_with_chdev_region`, όπου καλούμε την `unregister_chrdev_region` (`dev_no`, `linux_sensor_cnt`).

Με την `cdev_add(&linux_chrdev_cdev, dev_no, linux_minor_cnt)`, ο πυρήνας λαμβάνει το αρχικό `dev_t` και το πλήθος των `minor numbers` που οφείλει να χειριστεί ο driver. Έτσι, οι κλήσεις συστήματος για κάθε inode που ανήκει σε αυτό το εύρος θα αντιστοιχίζονται στα file operations όπως τα έχουμε ορίσει. Δίνεται τιμή στην `i_rdev` του inode.

Χρησιμοποιούμε τη βοηθητική συνάρτηση `linux_sensor_init` για κάθε συσκευή, `linux_sensor_init(&linux_sensors[i]);` η οποία:

- κάνει allocate μνήμη μιας σελίδας για κάθε μέτρηση στον πίνακα `msr_data`
- Αρχικοποιεί το `spinlock`
- Αρχικοποιεί το `waitqueue`

```
int linux_chrdev_init(void)
{
    /*
     * Register the character device with the kernel, asking for
     * a range of minor numbers (number of sensors * 8 measurements / sensor)
     * beginning with LINUX_CHRDEV_MAJOR:0
     */
    int ret;
    dev_t dev_no;
    unsigned int linux_minor_cnt = linux_sensor_cnt << 3;

    linux_sensors = kmalloc(sizeof(*linux_sensors) * linux_sensor_cnt, GFP_KERNEL);
    for (int i = 0; i < linux_sensor_cnt; i++)
        linux_sensor_init(&linux_sensors[i]);

    debug("initializing character device\n");
    cdev_init(&linux_chrdev_cdev, &linux_chrdev_fops);
    linux_chrdev_cdev.owner = THIS_MODULE;

    dev_no = MKDEV(LINUX_CHRDEV_MAJOR, 0);

    ret=register_chrdev_region(dev_no,linux_minor_cnt,"linux");

    if (ret < 0) {
        debug("failed to register region, ret = %d\n", ret);
        goto out;
    }

    ret=cdev_add(&linux_chrdev_cdev,dev_no,linux_minor_cnt);

    if (ret < 0) {
        debug("failed to add character device\n");
        goto out_with_chrdev_region;
    }
    debug("completed successfully\n");
    return 0;

out_with_chrdev_region:
    unregister_chrdev_region(dev_no, linux_minor_cnt);
out:
    return ret;
}
```

Syscall Open

Η open καλείται με την εντολή `cat /dev/<sensor+measurement>`.

Σκοπός της open είναι η προετοιμασία για ανάγνωση του stream του συγκεκριμένου αισθητήρα, ελέγχοντας αρχικά εάν η κλήση από τον χρήστη είναι έγκυρη (δηλαδή υπάρχει το ζητούμενο file για τον αισθητήρα όπως δημιουργήθηκε στην init) και ύστερα δημιουργώντας την αντίστοιχη δομή state.

Κάθε state είναι δείκτης που αποθηκεύεται στο `private_data` του δείκτη `filp`. Το πεδίο `void *private_data` ανήκει εξ ορισμού στο kernel driver interface και πάντα οφείλουμε να αποθηκεύουμε εκεί τη δομή state.

Εκτελείται η kernel function `ret = nonseekable_open(inode, filp)`, με την οποία “μαρκάρουμε” το αρχείο ως non-seekable, ώστε να επιτρέπεται μόνο η σειριακή ανάγνωση των δεδομένων.

Για να γίνει η ανάκτηση του αριθμού συσκευής και του είδους μέτρησης, λαμβάνουμε από το `inode` το `minor number`, στο οποίο τα 5 high bits δίνουν το `sensor_no`, ενώ τα 3 low bits το `measurement_no`.

Ομοίως, για να λάβουμε την αντίστοιχη δομή αισθητήρα, χρησιμοποιούμε το global array `linux_sensors[sensor_no]`, το οποίο αρχικοποιείται στην init.

Με την `state = kmalloc(sizeof(*state), GFP_KERNEL)` δημιουργείται νέο per-open state για κάθε αρχείο. Στη συνέχεια, το αρχικοποιούμε θέτοντας:

- `state->type = (enum linux_msr_enum)measurement_no;`
- `state->sensor = sensor;` // δείκτης στον αισθητήρα
- `state->buf_lim=0;` // `buf_lim` τα bytes έτοιμα για ανάγνωση
- `state->buf_timestamp = 0;`
- `sema_init(&state->lock,1);` // αρχικοποιούμε τον σηματοφόρο του state για χρήση ως mutex στην update

Σημειώνεται πως όταν κάνουμε `cat` μια συσκευή από 2 διαφορετικά terminals, δημιουργείται ένα νέο file και ένα νέο state για το ίδιο device `inode`.

```

static int linux_chrdev_open(struct inode *inode, struct file *filp)
{
    int ret;
    unsigned int minor;
    unsigned int sensor_no;
    unsigned int measurement_no;
    struct linux_chrdev_state_struct *state;
    struct linux_sensor_struct *sensor;
    unsigned long flags;
    debug("entering\n");
    ret = -ENODEV;
    if ((ret = nonseekable_open(inode, filp)) < 0)
        goto out;

    minor=iminor(inode);
    sensor_no=minor >> 3; //decode it with the high bits=sensor and low bits=measurement
    measurement_no=minor & 0x7;

    if (sensor_no>=linux_sensor_cnt) { //check that sensor exists
        ret=-ENODEV;
        goto out;
    }

    if(measurement_no>=LINUX_MSR_MAGIC) { //measurement is one of ours
        ret=-ENODEV;
        goto out;
    }

    sensor=&linux_sensors[sensor_no]; //get the struct for this sensor
    state=kmalloc(sizeof(*state),GFP_KERNEL);
    if (!state) {
        ret=-ENOMEM;
        goto out;
    }

    state->type=(enum linux_msr_enum)measurement_no; // which measurement
    state->sensor=sensor; //which sensor linux_sensor_init
    state->buf_lim=0; //no text yet
    state->buf_timestamp=0;
    sema_init(&state->lock,1); // semaphore to protect the buffer after fork or with threads

    filp->private_data=state;
    ret=0;

out:
    debug("leaving, with ret = %d\n", ret);
    return ret;
}

```

Syscall Read

Η read καλείται επίσης με την cat, αφού γίνει η αρχικοποίηση του state με την open. (Μία περίπτωση που η open τρέχει χωρίς να ακολουθήσει η read είναι η κλήση της `int fd = open("/dev/sensor0-temp", O_RDONLY);` σε κάποια διεργασία).

Όπως και στην open, ανακτούμε το sensor και το state με τη χρήση της `filp->private_data`. Χρησιμοποιούμε την `WARN_ON` για να ελέγξουμε ότι οι pointers που λαμβάνουμε δεν είναι null.

Εδώ εμφανίζεται το πρόβλημα του συγχρονισμού καθώς θέλουμε να προστατέψουμε τον buffer σε περίπτωση που πολλαπλά threads ή θυγατρικές διεργασίας μετά από `fork()` προσπαθούν να διαβάσουν ή να ανανεώσουν το buffer, ή εάν γίνει block στη read και χρειαστεί να μεταφερθεί στο waitqueue, οπότε και θα χρειαστεί να επαναληφθεί η read (**reentrant execution**). Χωρίς σημαφόρο, δύο reentrant executions μπορούν να μεταβάλλουν το state ταυτόχρονα.

Χρησιμοποιούμε την **down_interruptible** ώστε να μπορεί μια διεργασία στον χώρο χρήστη που περιμένει να λάβει τον σημαφόρο να διακοπεί.

Στη συνέχεια, εφόσον `f_pos = 0`, δηλαδή ο buffer είναι έτοιμος να γεμίσει με νέα δεδομένα, θα γίνει κλήση της `linux_chrdev_state_update(state)`, όπου γίνεται update εφόσον έχουν ληφθεί δεδομένα. Αλλιώς, θα κληθεί η **wait_event_interruptible**. Ελέγχεται η συνθήκη `linux_chrdev_needs_refresh(state)` και εάν είναι false, ο reader τοποθετείται στην waitqueue του αισθητήρα. Η update είναι η κλήση που αφυπνίζει τη διεργασία μόλις ληφθούν νέα δεδομένα.

Σε περίπτωση παραλληλισμού, η διεργασία/το νήμα απελευθερώνει τον σημαφόρο μετά την update, και το αποκτά εκ νέου πριν την εκτελέσει ξανά, ώστε να αποφευχθεί παράλληλη εκτέλεση της update και εγγραφούν λάθος δεδομένα στη δομή state.

Σημειώνεται ότι, μόλις ληφθούν νέα δεδομένα στο line discipline και περάσουν στο `/dev/tty`, ο πυρήνας θα μεταφερθεί σε interrupt context και θα κληθεί η `linux_sensor_update()` (στην `linux-sensors.c`), οπότε και θα μεταφερθούν τα δεδομένα στο αντίστοιχο buffer `msr_data` της μέτρησης του αντίστοιχου αισθητήρα. Ύστερα, αφυπνίζει τη διεργασία με την `wake_up_interruptible(&s->wq)`.

```

static ssize_t linux_chrdev_read(struct file *filp, char __user *usrbuf, size_t cnt, loff_t *f_pos)
{
    ssize_t ret = -ENODEV; // default error

    struct linux_sensor_struct *sensor;
    struct linux_chrdev_state_struct *state;
    state = filp->private_data;
    WARN_ON(!state);

    sensor = state->sensor;
    WARN_ON(!sensor);

    // semaphore to protect per-file cached buffer
    if (down_interruptible(&state->lock))
        return -ERESTARTSYS;
    /*
     * If the cached character device state needs to be
     * updated by actual sensor data (i.e. we need to report
     * on a "fresh" measurement, do so
     */
    if (*f_pos == 0) {
        while (linux_chrdev_state_update(state) == -EAGAIN) {
            up(&state->lock);
            if (wait_event_interruptible(sensor->wq,
                linux_chrdev_state_needs_refresh(state))) {
                ret = -ERESTARTSYS;
                goto out;
            }
            if (down_interruptible(&state->lock))
                return -ERESTARTSYS;
        }
    }
}

```

Για να γίνει update του buffer κάθε reader, χρησιμοποιούμε τη `loff_t *f_pos` του αρχείου, ως “εικονικός” δείκτης στον per-open buffer του αισθητήρα, ώστε να επιτραπεί η πρόσβαση από πολλαπλούς readers.

Εάν γίνει προσπάθεια για ανάγνωση πέραν του τέλους του αρχείου, επιστρέφεται 0. Αυτό μπορεί να γίνει εάν γίνεται ανάγνωση σε chunks και θέλουμε να την τερματίσουμε. Στη συνέχεια, γίνεται αντιγραφή των δεδομένων στο userspace με την `copy_to_user(usrbuf, state->buf_data + *f_pos, cnt)`. Ύστερα γίνεται update του `*f_pos` και επιστρέφουμε τον αριθμό των bytes που διαβάστηκαν `cnt`. Τέλος, γίνεται έλεγχος εάν βρισκόμαστε στο EOF, οπότε γίνεται auto-rewind, και στην επόμενη ανάγνωση θα κληθεί η update.


```

size_t available = state->buf_lim;

if (*f_pos >= available) { // EOF
    ret = 0;
    goto out;
}

/* Determine the number of cached bytes to copy to userspace */
if (cnt > available - *f_pos)
    cnt = available - *f_pos;

if (copy_to_user(usrbuf, state->buf_data + *f_pos, cnt)) {
    ret = -EFAULT;
    goto out;
}

*f_pos += cnt;
ret = cnt;

/* Auto-rewind on EOF mode */
if (*f_pos == available)
    *f_pos = 0;

out:
    up(&state->lock);
    return ret;
}

```

lunix_chrdev_state_update

Σκοπός είναι η ανανέωση του state της αντίστοιχης reader διεργασίας με τη λήψη τιμών από τους sensor msr_data buffers και η επιστροφή τιμών σε human-readable μορφή για την τύπωση στο terminal.

Με τη χρήση spinlocks, για να αποφευχθεί η εγγραφή “μπερδεμένων” δεδομένων (λ.χ. να εγγραφούν μερικώς raw τιμές προηγούμενης μέτρησης με timestamp καινούριας), γίνεται ανάγνωση του sensor msr_data buffer και λήψη των raw bytes και του timestamp της τελευταίας μέτρησης.

Για να στείλουμε τα δεδομένα στη σωστή μορφή για τύπωση, χρησιμοποιούμε lookup_table όπως έχει οριστεί στη **mk-lunix-lookup.c**. Εκεί γίνεται ο υπολογισμός των πραγματικών τιμών που αντιστοιχούν στα

αποσταλμένα bytes βάσει του πρωτοκόλλου των αισθητηρών. Ο λόγος που δεν μπορεί να γίνει στον πυρήνα, είναι επειδή δεν υποστηρίζει την αριθμητική κινήτης υποδιαστολής.

Υστερα, γίνεται έλεγχος εάν το δοσμένο state buf_timestamp είναι πιο πρόσφατο από το timestamp των δεδομένων που διαβάστηκαν, με τη χρήση της `linux_chrdev_needs_refresh`. Εάν δεν διαβάστηκαν νέα δεδομένα, επιστρέφεται error `-EAGAIN`. Στην αντίθετη περίπτωση, καλείται η `state->buf_lim = snprintf(state->buf_data, LINUX_CHRDEV_BUFSZ, "%ld.%03ld\n", value / 1000, value % 1000)`; για να μετατρέψουμε τα raw bytes σε μια human-readable τιμή πριν αποθηκευτούν στο buf_data. Τέλος, ανανεώνεται το timestamp του state.

```
static int linux_chrdev_state_update(struct linux_chrdev_state_struct *state)
{
    struct linux_sensor_struct __attribute__((unused)) *sensor;
    unsigned long flags;
    uint32_t raw, timestamp;
    //debug("leaving\n");
    long value=0;

    sensor = state->sensor;

    /*
     * Grab the raw data quickly, hold the
     * spinlock for as little as possible.
     */

    // irqsave locks and disables interrupts
    spin_lock_irqsave(&sensor->lock, flags);
    raw = sensor->msr_data[state->type]->values[0]; //lookup table
    timestamp = sensor->msr_data[state->type]->last_update;
    spin_unlock_irqrestore(&sensor->lock, flags); // unlock and enable interrupts
    /* Why use spinlocks? See LDD3, p. 119 */

    if (state->type == BATT) {
        value = lookup_voltage[raw];
    } else if (state->type == TEMP) {
        value = lookup_temperature[raw];
    } else if (state->type == LIGHT) {
        value = lookup_light[raw];
    } else {
        /* Fallback for safety */
        value = 0;
    }

    if (!linux_chrdev_state_needs_refresh(state))
        return -EAGAIN;

    /*
     * Now we can take our time to format them,
     * holding only the private state semaphore
     */
    state->buf_lim = snprintf(state->buf_data, LINUX_CHRDEV_BUFSZ,
                             "%ld.%03ld\n", value / 1000, value % 1000);
    state->buf_timestamp = timestamp;

    debug("leaving\n");
    return 0;
}
```

linux_chrdev_needs_refresh

Συγκρίνει το timestamp των πιο πρόσφατων δεδομένων του sensor buffer με το timestamp του state και επιστρέφει true εφόσον υπάρχουν νέα δεδομένα και οφείλουμε να ανανεώσουμε τη δομή state.

Στην read, εάν η κλήση της update στην read επιστρέφει -EAGAIN, δηλαδή δεν υπάρχουν νέα δεδομένα, καλείται η linux_chrdev_needs_refresh με την down_wait_interruptible, οπότε και η διεργασία προστίθεται σε waitqueue μέχρι να αφυπνιστεί από την linux_sensor_update() (στην linux-sensors.c).

```
static int linux_chrdev_state_needs_refresh(struct linux_chrdev_state_struct *state) {
    struct linux_sensor_struct *sensor;

    WARN_ON (!(sensor = state->sensor));

    return sensor->msr_data[state->type]->last_update > state->buf_timestamp;

    /* The following return is bogus, just for the stub to compile */
    return 0;
}
```

linux_chrdev_release

Εκτελείται κάθε φορά που ένα state κλείνει, δηλαδή γίνεται syscall close του per-open file του. Αποδεσμεύουμε τη δομή state και διαγράφουμε τον pointer filp->private_data.

```
static int linux_chrdev_release(struct inode *inode, struct file *filp)
{
    struct linux_chrdev_state_struct *state = filp->private_data;

    if (state)
        kfree(state);

    filp->private_data = NULL;
    return 0;
}
```

linux_chrdev_destroy

Εκτελείται με την `rmmmod`, δηλαδή όταν αφαιρείται ο οδηγός. Σκοπός είναι να διαγραφούν όλες οι δομές δεδομένων που χρησιμοποιήθηκαν για τον `driver` και όλη η μνήμη που είχε ανατεθεί σε αυτές. Συγκεκριμένα:

- `cdev_del`: διαγράφεται η δομή του αισθητήρα
- `unregister_chrdev_region`: αποδεσμεύονται όλα τα device nodes
- `linux_sensor_destroy`: καλεί την `free_page(s->msr_data[i])` για κάθε αισθητήρα
- `kfree(linux_sensors)`: αποδεσμεύεται ο πίνακας αισθητηρών

```
void linux_chrdev_destroy(void)
{
    dev_t dev_no;
    unsigned int linux_minor_cnt = linux_sensor_cnt << 3;

    debug("entering\n");
    dev_no = MKDEV(LINUX_CHRDEV_MAJOR, 0);
    cdev_del(&linux_chrdev_cdev);
    unregister_chrdev_region(dev_no, linux_minor_cnt);

    for (i = 0; i < linux_sensor_cnt; i++)
        linux_sensor_destroy(&linux_sensors[i]);

    kfree(linux_sensors);

    debug("leaving\n");
}
```